DressMe : l'architecture en bref

Les agents sont des **graphes LangGraph**. Ils echangent par le protocole **A2A**. Un **orchestrateur** choisit quel agent repond. Dans tous les agents, le LLM decide et redige ; le code verifie et calcule.

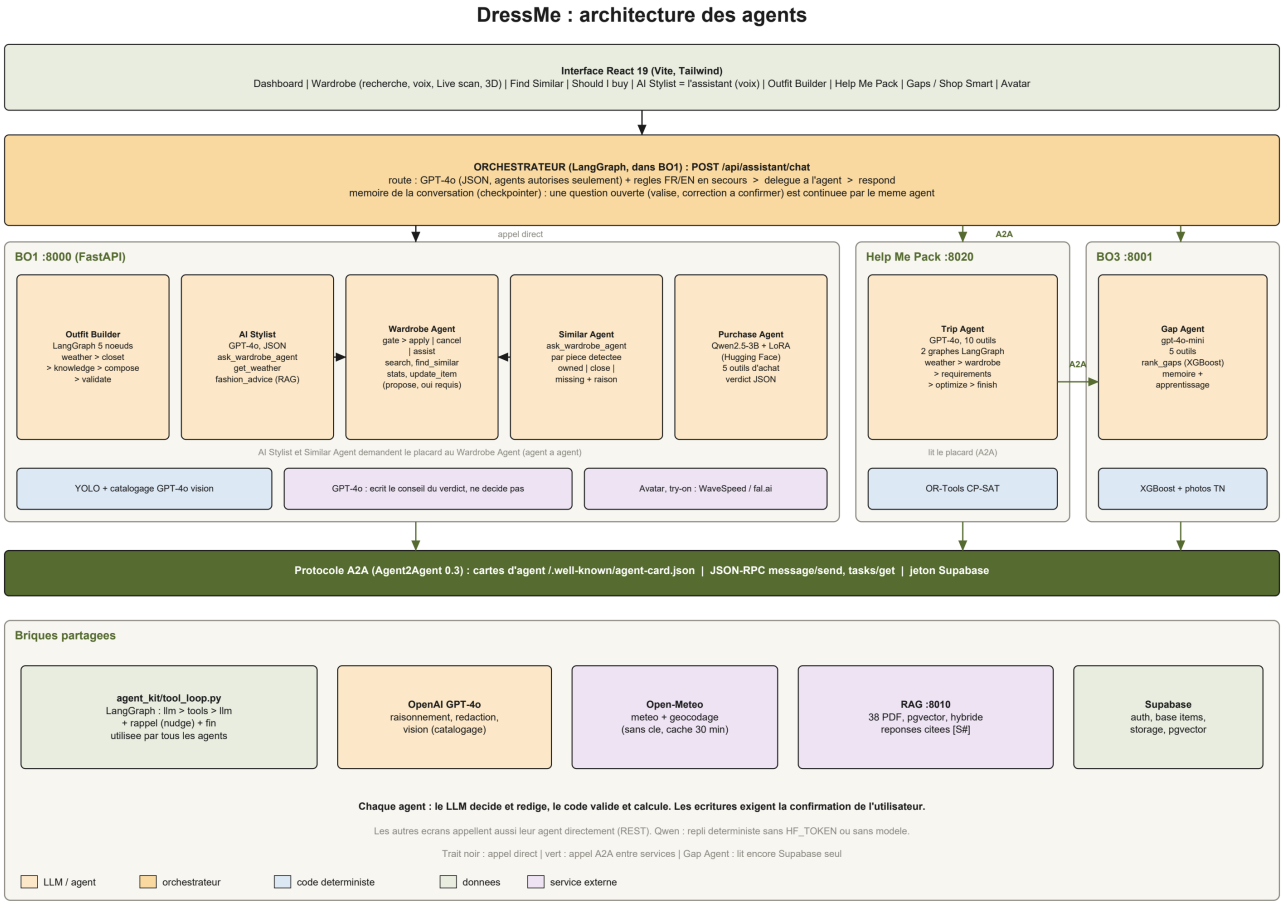


Figure : interface, orchestrateur, agents par service, protocole A2A et briques partagees.

1. L'orchestrateur

Role. C'est le chef d'orchestre : il recoit chaque message de la page AI Stylist (`POST /api/assistant/chat`), choisit l'agent qui doit repondre, lui delegue la demande et renvoie sa reponse. Il decide qui repond ; il ne repond pas lui-meme.

Noeud du graphe	Ce qu'il fait
route	GPT-4o choisit l'agent (seuls les agents de la liste sont acceptes). Si le LLM tombe, des regles FR/EN prennent le relais.
stylist | wardrobe | outfit | trip | gap	appelle l'agent choisi.
redirect	"Should I buy" et "Find Similar" ont besoin d'une photo : la reponse ouvre la bonne page.
clarify	demande ce que l'utilisateur veut quand la demande est trop vague.
respond	garde l'echange en memoire (un fil par utilisateur). Une question restee ouverte (valise, correction a confirmer) est reprise par le meme agent.

Comment il appelle les agents

Agent	Appel
AI Stylist, Wardrobe Agent, Outfit Builder	direct, dans le meme service (BO1)
Trip Agent (Help Me Pack), Gap Agent (BO3)	A2A avec le jeton de l'utilisateur : ce sont d'autres services

2. A2A : appeler un agent en trois temps

- **La carte** : `GET /a2a/<agent>/.well-known/agent-card.json` donne le nom, les competences et la securite de l'agent (publique).
- **La tache** : `POST /a2a/<agent>`, message JSON-RPC `message/send`, avec le jeton Supabase.
- **Le resultat** : une tache `completed`, `failed` ou `input-required`, avec un texte et des donnees. Pour continuer une question ouverte, on renvoie le meme `contextId`.

```
client = A2AClient(url, token) ; tache = client.send("Prepare ma valise pour Paris") ;
client.text_of(tache)
```

Chaque agent expose sa carte : `/a2a/stylist`, `/a2a/wardrobe`, `/a2a/similar`, `/a2a/outfit` (BO1), `/a2a/trip` (8020), `/a2a/gap` (BO3). Le Trip Agent appelle lui-meme le Wardrobe Agent (lire le placard) et le Gap Agent (lacunes) par A2A.

3. Comment chaque agent est construit

Tous suivent le meme modele. La boucle commune est dans `agent_kit/tool_loop.py` : `llm` → `tools` → `llm`, avec un rappel unique et une fin.

Agent	Construction
AI Stylist	boucle d'outils GPT-4o : ask_wardrobe_agent, get_weather (Open-Meteo), fashion_advice (RAG). Reponse JSON : choix, meteo, tenue, raisons, sources [S#].
Wardrobe Agent	graphe gate → apply | cancel | assist. 4 outils : recherche, pieces proches, statistiques, update_item. update_item propose seulement : le code n'ecrit qu'apres le oui de l'utilisateur.
Similar Agent	demande au Wardrobe Agent ce que l'utilisateur possede pour chaque piece d'un look ; juge owned / close / missing ; le code verifie que la piece citee existe.
Outfit Builder	graphe weather → closet → knowledge (RAG) → compose → validate. Le code rejette les tenues invalides et renvoie le LLM une fois.
Trip Agent	GPT-4o, 10 outils, deux graphes. Le solveur OR-Tools choisit la valise, pas le LLM.
Gap Agent	boucle de 5 outils. Des regles decident ce qui est une lacune ; XGBoost ne fait que les ordonner.
Purchase Agent	Qwen2.5-3B + LoRA (Hugging Face) pilote les outils d'achat et rend un verdict JSON verifie ; GPT-4o le reecrit seulement. Repli deterministe sans HF_TOKEN.

Briques partagees

- **agent_kit** (boucle LangGraph) et **a2a_protocol** (carte, serveur, client) : le code commun.
- **GPT-4o**, **Open-Meteo** (meteo, sans cle), le **RAG** (38 PDF de mode, reponses citees) et **Supabase** (auth, base, stockage).

A retenir

- L'orchestrateur est le seul a choisir qui repond. Les agents ne s'appellent entre eux que pour deux raisons voulues : demander le placard au Wardrobe Agent (AI Stylist, Similar Agent, Trip Agent) et, pour le Trip Agent, demander les lacunes au Gap Agent.
- Les ecritures exigent toujours la confirmation de l'utilisateur.
- Limites : le Purchase Agent n'est actif qu'avec HF_TOKEN et un modele executable ; le Gap Agent lit encore Supabase seul.